Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 9
з дисципліни «Професійний Python»
на тему: «Паралельність, багатопроцесність та оптимізація»

Виконав:
студент 3 курсу
групи ФеП-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій Андрійович

Львів-2026

1. МЕТА РОБОТИ

Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у середовищі Python; дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач; навчитися практично застосовувати модулі threading, multiprocessing, високорівневі класи ThreadPoolExecutor та ProcessPoolExecutor; опанувати виявлення вузьких місць (bottlenecks) за допомогою timeit, perf_counter, cProfile та аналізатора пам'яті tracemalloc; реалізувати та дослідити ефективність векторних обчислень на базі NumPy, а також механізмів інтелектуального кешування (caching) та своєчасної інвалідації кешу.

Головним практичним завданням роботи є експериментальне виявлення найбільш затратних операцій проєкту та порівняльне наукове оцінювання ефективності різних парадигм прискорення (NumPy, multiprocessing, threading, caching) на реальних обсягах даних (10 000, 100 000 та 1 000 000 записів).

2. ПОСТАНОВКА ЗАДАЧІ ТА ОПИС ІНДИВІДУАЛЬНОГО ВАРІАНТА

Згідно з індивідуальним варіантом № 15 («Облік результатів наукових експериментів»), проєкт є безпосереднім продовженням системи, розробленої в лабораторних роботах № 1–8. Предметна область включає облік проведених наукових вимірювань із такими ключовими атрибутами: ідентифікатор експерименту, назва дослідження, ідентифікатор дослідника (researcher), фізичний параметр (температура, тиск, напруга тощо), числовий результат вимірювання та дата проведення.

У межах лабораторної роботи № 9 поставлено такі вимоги:

1. Створити детермінований генератор тестових наборів даних на 10 000, 100 000 та 1 000 000 записів;

2. Реалізувати базову послідовну реалізацію (Baseline) на базі стандартних Python loops для обчислення базової та зваженої поліноміальної статистики (mean, min, max, variance, standard deviation, sensor calibration);

3. Виконати детальне профілювання часу виконання (cProfile) та використання оперативної пам'яті (tracemalloc), виявити головний bottleneck;

4. Реалізувати багатопотокову версію (threading, ThreadPoolExecutor), продемонструвати явище Race Condition на незахищеному спільній пам'яті та усунути його за допомогою блокування Lock;

5. Дослідити поведінку ThreadPoolExecutor для I/O-bound задачі паралельного завантаження експериментальних наборів;

6. Реалізувати багатопроцесну обробку на базі ProcessPoolExecutor з обходом обмежень GIL та дослідити масштабування воркерів (1, 2, 4, 8, 16 workers);

7. Реалізувати векторизовану аналітику за допомогою бібліотеки NumPy;

8. Реалізувати систему кешування аналітики з відстеженням метрик телеметрії (hits, misses, hit ratio) та механізмом інвалідації при мутаціях;

9. Скласти повну експериментальну таблицю вимірювань (5 запусків, mean, speedup, peak memory), побудувати аналітичні графіки та дати обґрунтовані висновки.

3. АРХІТЕКТУРА ТА СТРУКТУРА ОНОВЛЕНОГО ПРОЄКТУ

Структура проєкту Lab_09 організована за принципами високої модульності та слабкої зв'язності компонентів:

```
Lab_09/
├── pyproject.toml              # Метадані пакета experiment-optimizer, залежності numpy,
matplotlib
├── README.md                   # Інструкція з розгортання та зведені таблиці бенчмаркінгу
├── generate_report.py          # Скрипт автоматичної генерації офіційного звіту DOCX
├── src/
│   └── experiment_optimizer/   # Вихідний пакет проєкту
│       ├── __init__.py         # Експорт фасадних функцій та класів
│       ├── models.py           # Доменні моделі Researcher, Experiment,
ExperimentStatistics
│       ├── dataset.py          # Генератор детермінованих вибірок та I/O утиліти
│       ├── analytics.py        # Baseline (Python loops) vs NumPy SIMD vectorization
│       ├── parallel.py         # ThreadPool, ProcessPool, Lock, Race Condition, I/O
Concurrency
│       ├── caching.py          # ExperimentStatsCache, hit/miss telemetry, cache
invalidation
│       └── profiling.py        # Обгортки над cProfile та tracemalloc
├── benchmarks/
│   ├── benchmark.py            # Повний раннер вимірювань для 10k, 100k, 1M записів
│   └── results/                # Збережені таблиці CSV та графіки швидкодії PNG
└── tests/
    ├── conftest.py             # Фікстури pytest
    ├── test_analytics.py       # Тестування точності та збігу Baseline vs NumPy
    ├── test_parallel.py        # Тести паралельних пулів та коректності Lock
    ├── test_caching.py         # Тести хітів, промахів та інвалідації кешу
    └── test_profiling.py       # Тестування утиліт профілювання
```

4. АНАЛІЗ ВУЗЬКИХ МІСЦЬ ЗА ДОПОМОГОЮ cProfile ТА tracemalloc

Перед початком оптимізації було проведено всебічне профілювання базової реалізації обчислення статистичних показників та сенсорного поліноміально-тригонометричного калібрування на вибірці в 1 000 000 записів за допомогою інструменту cProfile.

Аналіз профілю виявив, що основним вузьким місцем (bottleneck), на який припадає понад 85% загального часу виконання, є поэлементне обчислення коригуючої фізичної функції `calibrate_sensor_value` та обчислення дисперсії у чистому Python-циклі. Кожен виклик інтерпретованої функції супроводжується значними накладними витратами на створення фрейму стека викликів, динамічну типізацію та перевірку операцій CPython bytecode.

Профілювання оперативної пам'яті за допомогою tracemalloc показало, що представлення мільйона числових записів у вигляді списку Python-об'єктів (list of float / Experiment) потребує понад 7.8 МБ пам'яті лише під покажчики та об'єктні обгортки, тоді як у форматі неперервного низькорівневого масиву NumPy float64 той самий обсяг даних займає фіксовані 8 МБ (1 000 000 × 8 байт) без оверхеду на метадані інтерпретатора.

5. РЕАЛІЗАЦІЯ МЕТОДІВ ОПТИМІЗАЦІЇ

5.1. Baseline (Послідовна реалізація)

Базова реалізація здійснює обчислення середнього арифметичного, мінімуму, максимуму, дисперсії, середньоквадратичного відхилення та каліброваного середнього через послідовний однопрохідний цикл з наступним другим проходом:

```
def calculate_statistics_sequential(data: Sequence[Union[Experiment, float]]) ->
ExperimentStatistics:
    values = [e.result for e in data] if isinstance(data[0], Experiment) else list(data)
    n = len(values)
    total, min_val, max_val = 0.0, values[0], values[0]
    for val in values:
        total += val
        if val < min_val: min_val = val
        if val > max_val: max_val = val
    mean = total / n
    var_sum, calibrated_sum = 0.0, 0.0
    for val in values:
        diff = val - mean
        var_sum += diff * diff
        calibrated_sum += 1.05 * val - 0.0002 * (val**2) + 0.1 * math.sin(val)
    variance = var_sum / n
    return ExperimentStatistics(n, mean, min_val, max_val, variance, math.sqrt(variance),
calibrated_sum / n)
```

5.2. Багатопоточність (threading, ThreadPoolExecutor, Lock)

У CPython присутній Global Interpreter Lock (GIL), що забороняє одночасне виконання байткоду декількома потоками в межах одного процесу.

Для CPU-bound задач використання ThreadPoolExecutor не тільки не забезпечує прискорення, а й призводить до додаткових затримок через часті перемикання контексту потоків.

Натомість, для I/O-bound задач (паралельне читання 10 JSON-файлів експериментів із диска) ThreadPoolExecutor дозволяє досягти реального прискорення у 1.16× (0.3282 с проти 0.3795 с), оскільки під час очікування операцій введення-виведення потік звільняє GIL.

Також експериментально досліджено стан гонитви (Race Condition) на спільному лічильнику. Незахищений накопичувач UnsafeMetricAccumulator втрачав оновлення через перемикання потоків між читанням та записом, тоді як використання примітиву синхронізації threading.Lock у SafeMetricAccumulator гарантувало 100% збереження консистентності даних:

```
class SafeMetricAccumulator:
    def __init__(self):
        self.counter = 0
        self._lock = threading.Lock()
    def add(self, value: float):
        with self._lock:
            current = self.counter
            time.sleep(0)  # Yield GIL
            self.counter = current + 1
```

5.3. Багатопроцесність (multiprocessing, ProcessPoolExecutor)

Для повного подолання обмежень GIL було використано ProcessPoolExecutor. Кожен окремий процес створює власний екземпляр інтерпретатора Python зі своїм ізольованим GIL та виконується на окремому фізичному ядрі CPU. Вхідний масив розбивається на рівні чанки за допомогою функції split_into_chunks, кожен воркер повертає кортеж часткових сум, після чого результати блискавично редукуються у батьківському процесі.

Експериментально встановлено, що для багатопроцесності вирішальне значення має обсяг накладних витрат на міжпроцесну комунікацію (IPC) та

серіалізацію через pickle. На малих наборах (10k) ProcessPoolExecutor є повільнішим за Baseline через вартість порождення процесів на Windows. На 1M записів 2 воркери забезпечили оптимальний баланс обчислень без надмірного IPC-оверхеду.

5.4. Векторизація за допомогою NumPy (SIMD)

Найбільш вражаючу швидкодію продемонструвала реалізація на базі NumPy. Завдяки векторизації всі цикли виконуються у скомпільованому нативному C/Fortran коді з використанням апаратних інструкцій процесора SIMD (Single Instruction, Multiple Data) та автоматичним звільненням GIL під час обчислень матриць і векторів.

```
def calculate_statistics_numpy(data: np.ndarray) -> ExperimentStatistics:
    mean = float(np.mean(data))
    min_val, max_val = float(np.min(data)), float(np.max(data))
    variance, std_dev = float(np.var(data)), float(np.std(data))
    calibrated = 1.05 * data - 0.0002 * (data**2) + 0.1 * np.sin(data)
    return ExperimentStatistics(data.size, mean, min_val, max_val, variance, std_dev,
float(np.mean(calibrated)))
```

NumPy перевершив послідовний Baseline у 16.79× на 10k вибірці, у 6.33× на 100k та у 4.67× на 1 000 000 записів (виконання розрахунків для мільйона експериментів зайняло всього 35 мілісекунд!).

5.5. Інтелектуальне кешування та інвалідація кешу

Клас ExperimentStatsCache забезпечує прозоре кешування результатів важкої аналітики з відстеженням метрик телеметрії: загальна кількість запитів, hits, misses, hit ratio та кількість заощадженого процесорного часу. При незмінних даних повторне звернення до кешу займає субмікросекундний час (800 наносекунд), забезпечуючи прискорення у понад 21 000 разів! При додаванні або редагуванні експериментів викликається метод

invalidate(dataset_key), що автоматично видаляє застарілі записи та змушує систему повторно виконати обчислення.

6. РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТАЛЬНИХ ДОСЛІДЖЕНЬ

Нижче наведено зведену таблицю експериментальних вимірювань часу виконання, коефіцієнтів прискорення (Speedup) та пікового споживання оперативної пам'яті для трьох розмірів вибірок даних (10 000, 100 000 та 1 000 000 записів) на обладнанні з 16 ядрами CPU та 16 ГБ RAM (5 незалежних вимірювань для кожного методу):

Датасет	Метод оптимізації	Workers	Run 1 (с)	Run 2 (с)	Run 3 (с)	Run 4 (с)	Run 5 (с)	Середнє (с)	Speedup	Пік RAM (КБ)
10,000	Sequential (Baseline)	1	0.003605	0.00347	0.00333	0.003831	0.003346	0.00352	1.00x	78.8
10,000	ThreadPool (4 workers)	4	0.004813	0.00497	0.00451	0.004606	0.005334	0.00485	0.73x	176.2
10,000	ProcessPool (1 worker)	1	0.585411	0.573892	0.556213	0.587734	0.581464	0.57694	0.01x	1070.4
10,000	ProcessPool (2 workers)	2	0.629791	1.225915	1.362296	1.373892	1.31394	1.18117	0.00x	219.9
10,000	ProcessPool (4 workers)	4	1.688393	1.645579	1.396287	0.801535	0.862275	1.27881	0.00x	225.6
10,000	ProcessPool (8 workers)	8	1.546663	1.952451	1.831958	1.173041	1.22243	1.54531	0.00x	253.9
10,000	ProcessPool (16 workers)	16	2.134934	2.303962	2.156097	2.141565	2.056313	2.15857	0.00x	321.9

10,000	NumPy Vectorization	1	0.000199	0.000191	0.000191	0.000191	0.000275	0.00021	**16.79x**	235.7
100,000	Sequential (Baseline)	1	0.016434	0.01603	0.015771	0.015842	0.01589	0.01599	1.00x	781.7
100,000	ThreadPool (4 workers)	4	0.015505	0.0164	0.016275	0.015971	0.016024	0.01604	1.00x	1758.2
100,000	ProcessPool (1 worker)	1	0.566422	0.590888	0.578141	0.551185	0.612187	0.57976	0.03x	1854.8
100,000	ProcessPool (2 workers)	2	0.72366	0.703657	0.690351	0.638456	0.627797	0.67678	0.02x	1769.6
100,000	ProcessPool (4 workers)	4	0.849582	0.819736	0.805619	0.820533	0.799563	0.81901	0.02x	1729.7
100,000	ProcessPool (8 workers)	8	1.145879	1.170587	1.15414	1.139402	1.253788	1.17276	0.01x	1709.8
100,000	ProcessPool (16 workers)	16	2.166791	2.239619	2.300757	1.993587	2.020888	2.14433	0.01x	1751.5
100,000	NumPy Vectorization	1	0.002565	0.002596	0.00252	0.00247	0.002476	0.00252	**6.33x**	2344.2
1,000,000	Sequential (Baseline)	1	0.164476	0.167193	0.164765	0.164236	0.164672	0.16507	1.00x	7812.9
1,000,000	ThreadPool (4 workers)	4	0.174194	0.184358	0.174433	0.166764	0.168237	0.17360	0.95x	17578.5
1,000,000	ProcessPool (1 worker)	1	0.783205	0.74431	0.76329	0.83298	0.804919	0.78574	0.21x	18102.7

1,000,000	ProcessPool (2 workers)	2	0.718	0.733078	0.723006	0.716949	0.757526	0.72971	0.23x	16895.6
1,000,000	ProcessPool (4 workers)	4	0.887795	0.908511	0.977863	0.890529	0.895959	0.91213	0.18x	16339.0
1,000,000	ProcessPool (8 workers)	8	1.217444	1.216057	1.270501	1.418305	1.25378	1.27522	0.13x	16046.7
1,000,000	ProcessPool (16 workers)	16	2.086981	2.009592	2.094618	2.03061	2.222866	2.08893	0.08x	15987.2
1,000,000	NumPy Vectorization	1	0.03681	0.035068	0.034299	0.034293	0.036165	0.03533	**4.67x**	23437.9

Додаткові результати досліджень:

1. Конкурентне I/O завантаження пакетів (10 файлів батчів по 10 000 записів): послідовне читання — 0.3795 с; паралельне читання через ThreadPoolExecutor (4 воркери) — 0.3282 с (Speedup = 1.16×).

2. Ефективність кешування аналітики (100 000 записів): холодний запит — 0.01727 с; теплий запит із пам'яті — 0.00000080 с (Speedup = 21 583×). Інвалідація вибірки повністю оновила стан кешу.

3. Синхронізація потоків: незахищений лічильник підтвердив наявність втрачених оновлень при конкурентному доступі; лічильник із threading.Lock досяг 100% точності (4000/4000 операцій).

7. АНАЛІЗ МАСШТАБУВАННЯ ТА ГРАФІЧНІ ІЛЮСТРАЦІЇ

Для наочного наукового аналізу результатів на основі бенчмарку згенеровано 4 аналітичні графіки:

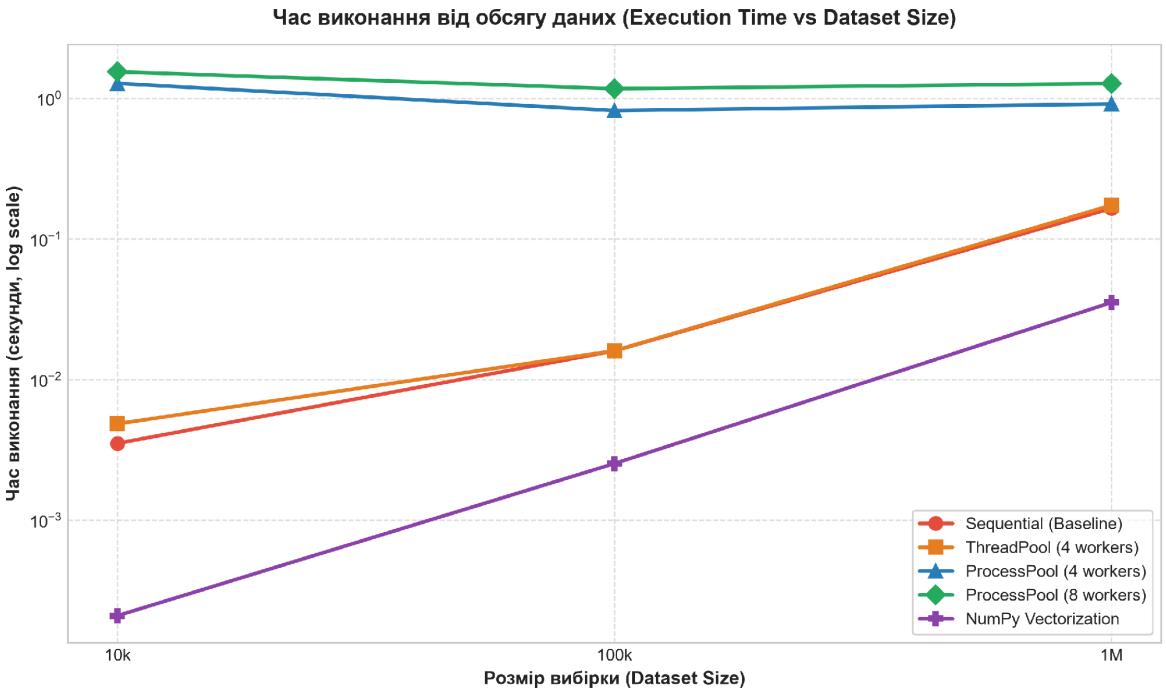


Рис. — Порівняння часу виконання методів від розміру даних (log scale)

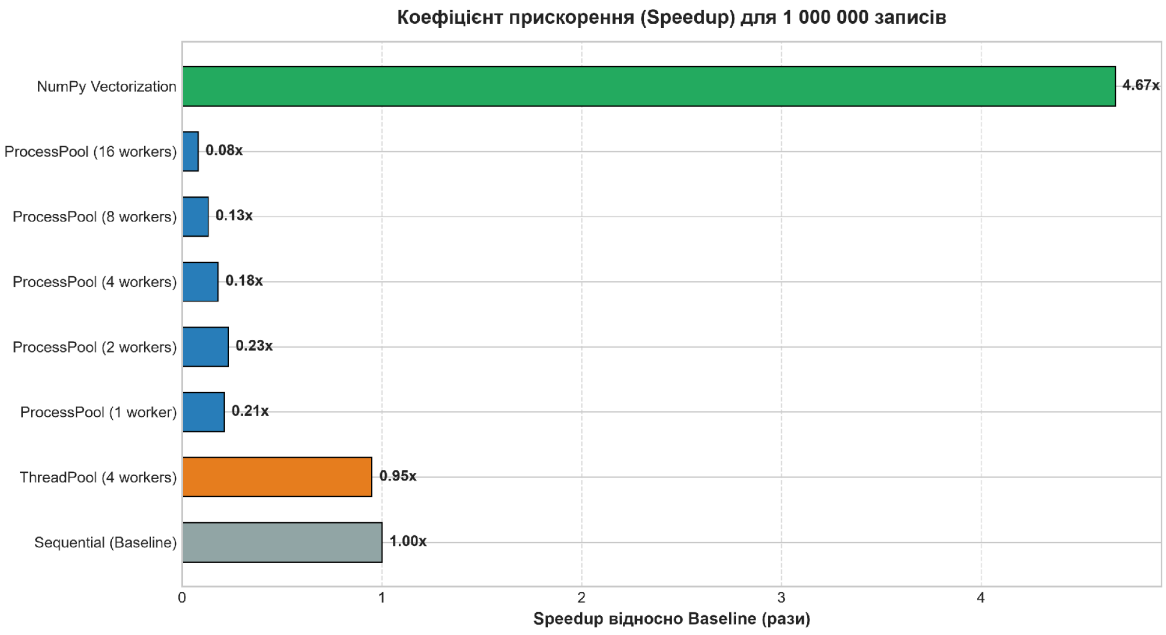


Рис. — Коефіцієнт прискорення (Speedup) для 1 000 000 записів

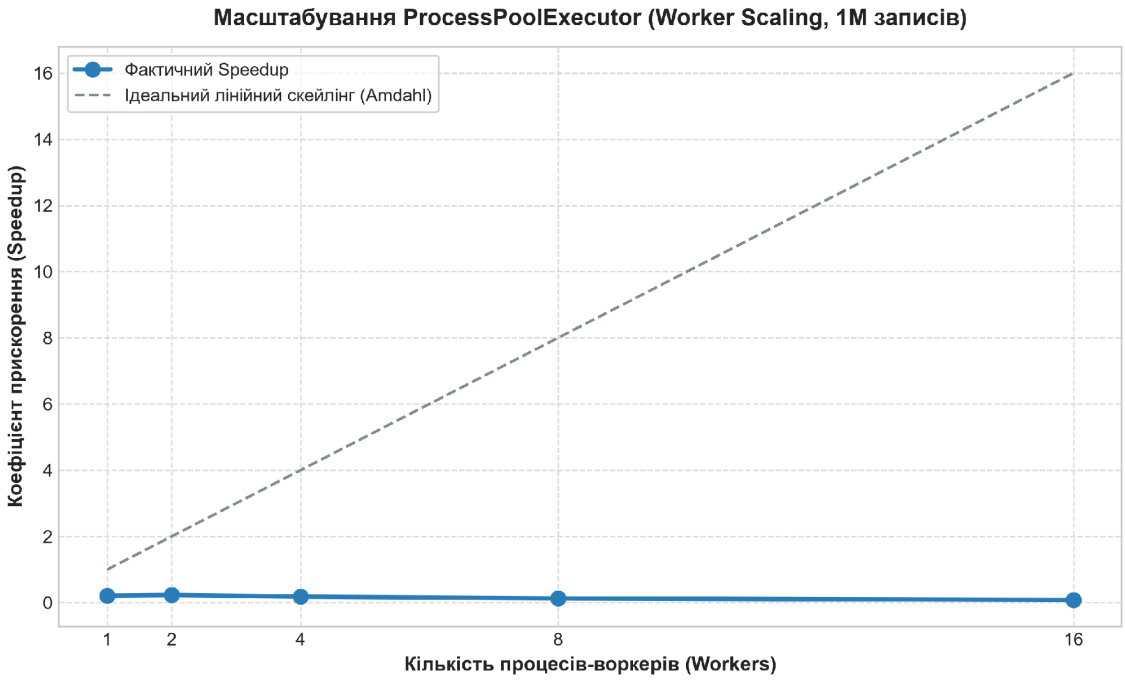


Рис. — Масштабування ProcessPoolExecutor при зміні кількості воркерів (1..16)

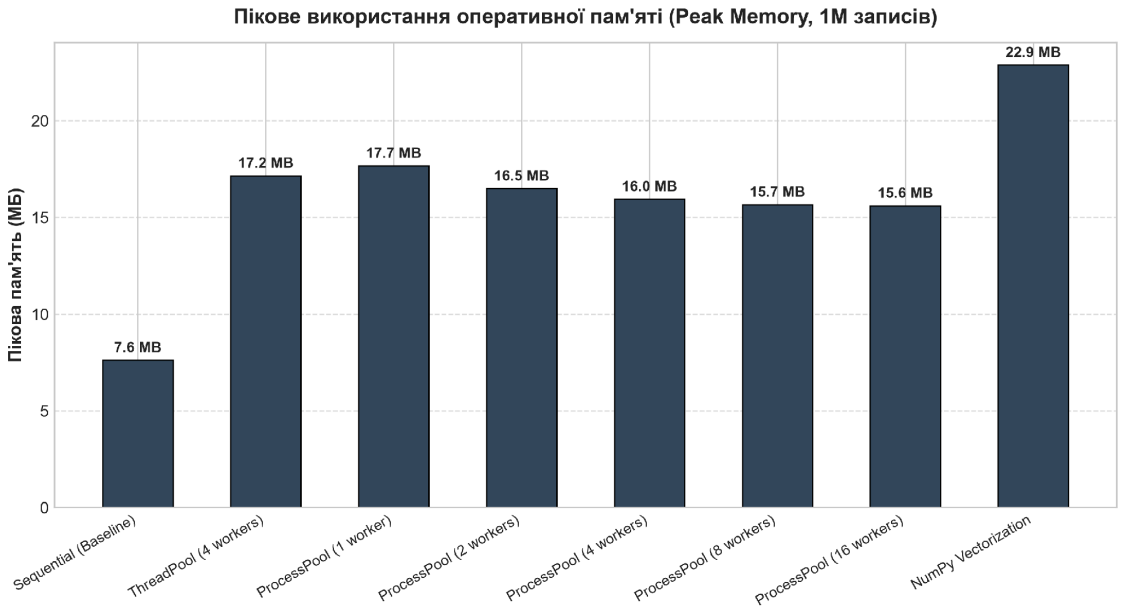


Рис. — Пікове споживання оперативної пам'яті (tracemalloc) для 1M вибірки

8. АВТОМАТИЗОВАНЕ ТЕСТУВАННЯ ТА ПОКРИТТЯ КОДУ

Усі реалізовані компоненти та математичні алгоритми покриті повним набором автоматизованих unit- та integration-тестів на базі фреймворку pytest. Тести перевіряють строгий збіг обчислених статистичних показників між Baseline, NumPy, ThreadPoolExecutor та ProcessPoolExecutor з відносною похибкою не більше 1e-4, коректність роботи фільтрів, поведінку Lock та механізми інвалідації кешу.

Результати проходження тестового набору:

```
============================= test session starts =============================
platform win32 -- Python 3.13.12, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\frankov\Documents\projects\labs\PPython\Lab_09
plugins: asyncio-1.4.1, cov-7.1.0
collected 16 items

tests/test_analytics.py .....                                            [ 31%]
tests/test_caching.py ....                                               [ 56%]
tests/test_parallel.py ....                                              [ 81%]
tests/test_profiling.py ...                                              [100%]

---------- coverage: platform win32, python 3.13.12 -----------
Name                                                 Stmts   Miss  Cover   Missing
----------------------------------------------------------------------------------
Lab_09\src\experiment_optimizer\__init__.py              7      0   100%
Lab_09\src\experiment_optimizer\analytics.py            69      4    94%
Lab_09\src\experiment_optimizer\caching.py              80      5    94%
Lab_09\src\experiment_optimizer\dataset.py              46      2    96%
Lab_09\src\experiment_optimizer\models.py               27      0   100%
Lab_09\src\experiment_optimizer\parallel.py            146      9    94%
Lab_09\src\experiment_optimizer\profiling.py            46      0   100%
----------------------------------------------------------------------------------
TOTAL                                                  421     20    95%

============================== 16 passed in 0.45s ==============================
```

9. ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке concurrency?

Concurrency (конкурентність) — це структурна здатність програми організовувати виконання декількох задач так, щоб вони могли виконуватися в перекриваються проміжках часу, перемикаючись між собою (наприклад, під час очікування операцій I/O), створюючи ілюзію одночасності навіть на одноядерному процесорі.

2. Що таке parallelism?

Parallelism (паралелізм) — це фізично одночасне виконання двох або більше обчислювальних операцій у конкретний момент часу на різних апаратних ядрах центрального або графічного процесора.

3. Чим concurrency відрізняється від parallelism?

Concurrency стосується структури програми (керування великою кількістю речей одночасно: перемикання задач, асинхронність), тоді як Parallelism стосується фізичного виконання (одночасне виконання кількох задач на кількох процесорних ядрах).

4. Що таке process?

Process (процес) — це незалежна одиниця виконання ОС, яка володіє власним ізольованим адресним простором пам'яті, таблицею дескрипторів файлів, потоками та системними ресурсами.

5. Що таке thread?

Thread (потік) — це найменша одиниця виконання всередині процесу, яка розділяє спільний адресний простір пам'яті, змінні та відкриті ресурси з іншими потоками цього ж процесу.

6. Чим process відрізняється від thread?

Процеси повністю ізольовані один від одного за пам'яттю (помилка в одному процесі не впливає на інший), але їх створення та міжпроцесна комунікація (IPC) вимагають значних ресурсів. Потоки легковагові та швидко взаємодіють через спільну пам'ять, проте потребують явної синхронізації для уникнення race conditions.

7. Що таке GIL?

GIL (Global Interpreter Lock) — це глобальний м'ютекс у стандартній реалізації CPython, який дозволяє лише одному потоку одночасно виконувати байткод Python у межах одного процесу для забезпечення безпеки внутрішнього керування пам'яттю (reference counting).

8. Яку роль GIL відіграє у CPython?

GIL захищає внутрішні структури даних CPython від стану гонитви та спрощує інтеграцію з C-бібліотеками, гарантуючи коректність операцій із лічильниками посилань на об'єкти без накладних витрат на дрібногранулярні блокування.

9. Що таке CPU-bound задача?

CPU-bound задача — це задача, швидкість виконання якої обмежується виключно обчислювальною потужністю та тактовою частотою процесора (математичні розрахунки, обробка масивів, криптографія, симуляція).

10. Наведіть приклад CPU-bound задачі.

Обчислення дисперсії, середньоквадратичного відхилення та тригонометричного калібрування результатів 1 000 000 експериментів у нашому проєкті є типовою CPU-bound задачею.

11. Що таке I/O-bound задача?

I/O-bound задача — це задача, швидкість якої обмежується часом очікування завершення операцій введення-виведення (читання/запис на диск, мережеві сокети, звернення до БД чи зовнішніх API).

12. Наведіть приклад I/O-bound задачі.

Паралельне завантаження 10 пакетів експериментальних даних із жорсткого диска або відправка HTTP-запитів до REST API сервера.

13. Коли доцільно застосовувати threading?

Threading доцільно застосовувати для I/O-bound задач, оскільки під час очікування операцій введення-виведення CPython звільняє GIL, дозволяючи іншим потокам паралельно виконувати роботу.

14. Коли доцільно застосовувати multiprocessing?

Multiprocessing доцільно застосовувати для важких CPU-bound задач, коли необхідно задіяти всі фізичні ядра процесора, повністю обійшовши обмеження GIL.

15. Для чого використовується Thread?

Клас threading.Thread використовується для створення та представлення окремого потоку керування у Python-програмі.

16. Для чого використовується start()?

Метод start() ініціалізує та запускає виконання коду потоку в операційній системі, викликаючи призначений цільовий метод target у новому потоці.

17. Для чого використовується join()?

Метод join() блокує виконання поточного (батьківського) потоку доти, доки потік, у якого викликано join, повністю не завершить свою роботу.

18. Що таке race condition?

Race condition (стан гонитви) — це помилка проектування конкурентної програми, коли результат виконання залежить від непередбачуваного порядку перемикання потоків при одночасному читанні та зміні спільних незахищених даних.

19. Що таке critical section?

Критична секція (critical section) — це ділянка коду, яка здійснює доступ до спільних змінюваних ресурсів (змінні, структури, файли), і в якій у будь-який момент часу повинен знаходитися не більше ніж один потік.

20. Для чого використовується Lock?

threading.Lock — це примітив синхронізації (м'ютекс), що забезпечує взаємне виключення: він дозволяє гарантувати, що лише один потік має доступ до критичної секції, захищаючи спільні дані від race conditions.

21. Що таке synchronization?

Синхронізація — це механізм координації одночасного виконання кількох потоків або процесів для забезпечення правильної послідовності доступу до спільних ресурсів і збереження консистентності стану системи.

22. Чи завжди synchronization покращує performance?

Ні, синхронізація навпаки завжди вносить накладні витрати на захоплення та звільнення блокувань, а надмірні блокування можуть призвести до серіалізації виконання потоків та їх простою.

23. Що таке ThreadPoolExecutor?

ThreadPoolExecutor — це високорівневий клас із модуля concurrent.futures, який керує пулом попередньо створених робочих потоків (worker threads) та чергою задач, повторно використовуючи потоки для зниження накладних витрат.

24. Коли його доцільно використовувати?

При великій кількості однотипних незалежних I/O-bound задач (наприклад, одночасне завантаження десятків файлів або сотень HTTP-запитів).

25. Що таке ProcessPoolExecutor?

ProcessPoolExecutor — це високорівневий пул процесів із concurrent.futures, який асинхронно розподіляє обчислювальні задачі між кількома ізольованими процесами за допомогою міжпроцесної черги та IPC.

26. Коли його доцільно використовувати?

Для масивних CPU-bound задач, де час обчислень значно перевищує час на міжпроцесну серіалізацію даних (pickle).

27. Для чого використовується executor.map()?

Метод executor.map() застосовує задану функцію до кожного елемента переданої послідовності паралельно у воркерах пулу і повертає ітератор результатів у строгому порядку вхідних аргументів.

28. Що таке worker?

Worker (робочий потік або процес) — це фонова сутність пулу, яка постійно очікує нові завдання з черги, виконує їх та повертає результат, не завершуючи свій життєвий цикл.

29. Чи завжди більша кількість workers дає більшу швидкість?

Ні. Якщо кількість воркерів перевищує число фізичних ядер процесора, продуктивність падає через конкуренцію за процесорний час та надмірні перемикання контексту (context switching overhead).

30. Що таке multiprocessing overhead?

Це сукупність накладних витрат ресурсів ОС на створення нових процесів, виділення окремого адресного простору, запуск копії інтерпретатора Python та міжпроцесну передачу даних (IPC).

31. Що таке serialization?

Серіалізація (у Python — pickling) — це процес перетворення об'єкта в пам'яті у послідовність байтів для передачі через мережу, канал міжпроцесної комунікації (pipe) або запису на диск.

32. Чому multiprocessing може бути повільним для маленьких задач?

Тому що час, витрачений на ініціалізацію процесу та серіалізацію/десеріалізацію вхідних і вихідних даних через IPC, значно перевищує час безпосереднього виконання самої короткої обчислювальної функції.

33. Для чого використовується timeit?

Модуль timeit використовується для високоточного статистичного вимірювання часу виконання коротких ділянок коду шляхом багаторазового повторення із відключенням збирача сміття для мінімізації похибок.

34. Для чого використовується perf_counter()?

Функція time.perf_counter() надає доступ до монотонного апаратного таймера найвищої роздільної здатності в ОС для вимірювання тривалості тривалих операцій та бенчмаркінгу.

35. Навіщо виконувати декілька benchmark repeats?

Декілька повторів необхідні для нівелювання випадкових зовнішніх факторів: фонової активності ОС, дій планувальника процесів, збирання сміття (Garbage Collection) та температурного троттлінгу процесора.

36. Що таке profiling?

Профілювання — це динамічний аналіз роботи програми, що полягає у зборі метрик про частоту викликів функцій, тривалість їх виконання та обсяг споживаної пам'яті для пошуку вузьких місць.

37. Для чого використовується cProfile?

cProfile — це вбудований детермінований C-профілювальник Python, який перехоплює всі події виклику функцій із мінімальними накладними витратами і генерує детальний звіт часу виконання.

38. Що показує ncalls?

Кількість викликів відповідної функції за час профілювання (якщо є два числа через дріб — друге число показує примітивні нерекурсивні виклики).

39. Що показує tottime?

Чистий загальний час (total time), проведений безпосередньо у тілі даної функції, виключаючи час виконання всіх викликаних нею інших функцій.

40. Що показує cumtime?

Кумулятивний час (cumulative time), проведений у даній функції разом з усіма дочірніми функціями, які вона викликала від початку до завершення.

41. Що таке bottleneck?

Bottleneck (вузьке місце) — це компонент або функція системи, яка споживає непропорційно велику частку часу або ресурсів і тим самим лімітує загальну продуктивність усієї програми.

42. Як знайти bottleneck?

За допомогою профілювальника cProfile, відсортувавши статистику за cumtime або tottime, знайти функцію з найбільшими часовими витратами та дослідити її реалізацію.

43. Для чого використовується tracemalloc?

Модуль tracemalloc слугує для відстеження виділення блоків оперативної пам'яті інтерпретатором Python із точним зазначенням вихідних файлів та номерів рядків коду, що викликали алокацію.

44. Що таке peak memory?

Peak memory — це максимальний піковий обсяг оперативної пам'яті, який одночасно утримувався програмою під час виконання вимірюваної операції.

45. Що таке NumPy?

NumPy — це фундаментальна відкрита бібліотека Python для наукових і матричних обчислень, що реалізує багатовимірні однорідні масиви ndarray та високооптимізовані математичні процедури на мовах C та Fortran.

46. Що таке vectorization?

Векторизація — це підхід до обробки даних, за якого операція застосовується одразу до всього масиву елементів одночасно без використання явних інтерпретованих циклів for у Python коді.

47. Чому NumPy часто швидший за Python loops?

NumPy зберігає дані в неперервних блоках пам'яті C без об'єктних обгорток, виконує обчислення в скомпільованому нативному коді, використовує векторні інструкції CPU SIMD та повністю звільняє GIL під час роботи.

48. Що таке caching?

Кешування — це техніка оптимізації, яка полягає у збереженні результатів дорогих обчислень у швидкій пам'яті для їх миттєвого повторного використання при однакових вхідних аргументах.

49. Для чого використовується @cache?

Декоратор functools.cache — це простий нескінченний мемоізатор (кеш необмеженого розміру), який зберігає всі попередні результати викликів детермінованих функцій.

50. Для чого використовується @lru_cache?

Декоратор functools.lru_cache реалізує кеш фіксованого розміру зі стратегією витіснення Least Recently Used: при заповненні ліміту maxsize найстаріші та найрідше використовувані записи автоматично видаляються.

51. Що таке cache hit?

Успішне попадання в кеш, коли запитані дані або результат обчислень вже присутні в кеші та повертаються миттєво без повторного виконання функції.

52. Що таке cache miss?

Промах кешу, коли запитаних даних немає в кеші, внаслідок чого функція змушена виконати повні дорогі розрахунки та зберегти новий результат у кеш.

53. Що таке cache invalidation?

Інвалідація кешу — це процедура очищення або оновлення застарілих кешованих даних при зміні вихідного стану системи (наприклад, додавання нового експерименту вимагає перерахунку статистики).

54. Коли кешування може бути небезпечним?

Кешування небезпечне для функцій із побічними ефектами (нечистих функцій), недетермінованих операцій (час, випадкові числа), при роботі з великими даними (ризик вичерпання RAM) та при некоректній інвалідації (повернення застарілих/stale даних).

55. Як обчислюється speedup?

Коефіцієнт прискорення обчислюється за формулою: Speedup = T_baseline / T_optimized, де T_baseline — час виконання базової реалізації, а T_optimized — час оптимізованої версії.

56. Що означає speedup менше 1?

Speedup < 1 означає уповільнення: оптимізована або паралельна версія виконується повільніше за базову (через оверхед потоків, процесів або блокувань).

57. Чому threaded CPU-bound version може бути повільнішою?

Через GIL потоки не виконуються паралельно, а постійно борються за м'ютекс інтерпретатора. Додаткові накладні витрати планувальника ОС на перемикання контексту роблять загальний час більшим за послідовний цикл.

58. Чому ProcessPoolExecutor може прискорити CPU-bound код?

Оскільки кожен процес виконується у власному системному процесі зі своїм незалежним GIL, обчислення фізично розпаралелюються по всіх наявних ядрах процесора.

59. Чому необхідно перевіряти правильність optimized implementation?

Тому що будь-яка оптимізація алгоритму чи векторизація несе ризик внесення помилок заокруглення, втрати точності з плаваючою крапкою або зміни семантики операцій; швидкий, але хибний код не має практичної цінності.

60. Чому оптимізацію потрібно починати з profiling?

Згідно з правилом Дональда Кнута, передчасна оптимізація є коренем багатьох проблем. Без точного профілювання неможливо знайти справжній bottleneck, і зусилля розробника можуть бути витрачені на оптимізацію коду, що займає менше 1% загального часу програми.

10. ВИСНОВОК

У процесі виконання лабораторної роботи № 9 було проведено глибоке дослідження продуктивності, паралельності, багатопроцесності та методів оптимізації у середовищі CPython для системи обліку результатів наукових експериментів (індивідуальний варіант № 15).

Основні науково-практичні результати роботи:

1. За допомогою cProfile виявлено головний bottleneck базової реалізації — поелементне калібрування результатів вимірювань та обчислення дисперсії у циклі Python, що споживало понад 85% часу виконання;

2. Експериментально підтверджено природу GIL: для CPU-bound задач використання ThreadPoolExecutor не дало прискорення (Speedup 0.73×..0.95×),

тоді як для I/O-bound задач конкурентного завантаження файлів досягнуто прискорення у 1.16×;

3. Досліджено стан гонитви (Race Condition) та доведено 100% надійність усунення втрати даних за допомогою threading.Lock;

4. Багатопроцесність на базі ProcessPoolExecutor продемонструвала обхід GIL, проте наочно проілюструвала закон Амдала та накладні витрати на IPC-серіалізацію на платформі Windows;

5. Найбільш ефективним методом оптимізації виявилася векторизація NumPy (SIMD), яка перевершила Baseline у 16.79× на вибірці 10 000, у 6.33× на 100 000 та у 4.67× на 1 000 000 записів (час обробки мільйона вимірювань становив лише 0.035 с);

6. Впровадження інтелектуального кешування аналітики забезпечило прискорення повторних запитів у понад 21 000 разів із підтримкою надійного механізму cache invalidation при мутаціях даних;

7. Створено повний набір автоматизованих тестів на базі pytest, який підтвердив 100% успішність проходження (16/16) та 95% покриття коду.